

Unit - 3

Database creation using SQL

A database is a structured collection of data that is stored in a computer system. They are used to store and retrieve the data efficiently. Databases can be created using different query languages, and **SQL** is one such language.

The database developers and the users use this statement in SQL for creating the new database in the database systems. It creates the database with the name which has been specified in the Create Database statement.

Syntax of Create Database statement in SQL

1. **CREATE DATABASE** Database_Name;

In this syntax, **Database_Name** specifies the name of the database which we want to create in the system. We have to type the database name in query just after the 'Create Database' keyword.

Following are the most important points which are required to learn while creating a database:

- The database we want to create should be a simple and unique name, which can be easily identified.
- Database name should be no more than 128 characters.

Syntax of Create Database statement in MySQL

The same command is used in MySQL to create the new database for storing the structured data.

1. **CREATE DATABASE** Database_Name;

Syntax of Create Database in Oracle

There is no need to create the database in Oracle systems. In the Oracle database, we can directly create the database tables.

Examples of Create Database statement in SQL

In this article, we took the following two examples which will help how to run and perform the Create Database query in SQL:

Example 1:

This example creates the **Student** database. To create the Student database, you have to type the following command in Structured Query Language:

1. **CREATE DATABASE** Student ;

When this query is executed successfully, then it will show the following output:

Database created successfully

You can also verify that your database is created in SQL or not by using the following query:

1. **SHOW DATABASE** ;

SQL does not allow developers to create the database with the existing database name. Suppose if you want to create another Student database in the same database system, then the Create Database statement will show the following error in the output:

Can't create database 'Student'; database exists

So, firstly you have to delete the existing database by using the Drop Statement. You can also replace the existing database with the help of Replace keyword.

If you want to replace the existing Student database, then you have to type the following SQL query:

1. **CREATE OR REPLACE DATABASE** Student ;

Example 2:

Suppose, we want to create the Employee database in the system.

Firstly, we have to type the following command in Structured Query Language:

1. **CREATE DATABASE** Employee ;

When this query is executed successfully, then it will show the following output:

Database created successfully

You can also check that your database is created in SQL by typing the following query:

1. SHOW **DATABASE** ;

We know that SQL does not allow developers to create the database with the existing database name.

Suppose, we want to create another Employee database in the same database system, firstly, we have to delete the existing database using a drop statement, or we have to replace the existing Employee database with the help of the 'replace' keyword.

To replace the existing Employee database with a new Employee database, we have to type the following query in SQL:

1. **CREATE OR REPLACE DATABASE** Employee;

SQL commands – DDL & DML

DDL stands for Data Definition Language and refers to SQL commands used to create, modify, and delete database structures such as tables, indexes, and views. DML stands for Data Manipulation Language and refers to SQL commands used to insert, update, and delete data within a database.

DDL Commands in SQL

DDL (Data Definition Language) is a type of SQL command used to define data structures and modify data. It creates, alters, and deletes database objects such as tables, views, indexes, and users. Examples of DDL statements include CREATE, ALTER, DROP and TRUNCATE.

DML Commands in SQL

DML (Data Manipulation Language) is a type of SQL command used to manipulate data in a database. It inserts, updates, and deletes data from a database table. Examples of DML statements include INSERT, UPDATE, and DELETE.

Types of DDL Statements

1. **CREATE**: It is used to create objects in the database, such as tables, views, stored procedures, and more.
2. **ALTER**: It is used to modify the structure of an existing database object.
3. **DROP**: It is used to delete an entire object or part of an object from the database.
4. **TRUNCATE**: Used to delete all records from a table but does not delete the table structure.
5. **RENAME**: Used to rename an existing database object.

a. **CREATE** It is used to create a new table in the database.

Syntax:

1. CREATE TABLE TABLE_NAME (COLUMN_NAME DATATYPES[.....]);

Example:

1. CREATE TABLE EMPLOYEE(Name VARCHAR2(20), Email VARCHAR2(100), DOB DATE);

b. DROP: It is used to delete both the structure and record stored in the table.

Syntax

1. DROP TABLE table_name;

Example

1. DROP TABLE EMPLOYEE;

c. ALTER: It is used to alter the structure of the database. This change could be either to modify the characteristics of an existing attribute or probably to add a new attribute.

Syntax:

To add a new column in the table

1. ALTER TABLE table_name ADD column_name COLUMN-definition;

To modify existing column in the table:

1. ALTER TABLE table_name MODIFY(column_definitions....);

EXAMPLE

1. ALTER TABLE STU_DETAILS ADD(ADDRESS VARCHAR2(20));
2. ALTER TABLE STU_DETAILS MODIFY (NAME VARCHAR2(20));

d. TRUNCATE: It is used to delete all the rows from the table and free the space containing the table.

Syntax:

1. TRUNCATE TABLE table_name;

Example:

1. TRUNCATE TABLE EMPLOYEE;

Types of DML Statements

1. **INSERT:** Used to add new records to a database table.
2. **UPDATE:** Used to modify existing records in a database table.
3. **DELETE:** Used to delete existing records from a database table.
4. **MERGE:** Used to combine data from two or more tables into one.

5. **SELECT:** Used to retrieve data from one or more tables in a database.
6. **CALL:** Used to call a stored procedure or function.

a. INSERT: The INSERT statement is a SQL query. It is used to insert data into the row of a table.

Syntax:

1. INSERT INTO TABLE_NAME
2. (col1, col2, col3,.... col N)
3. VALUES (value1, value2, value3, valueN);

Or

1. INSERT INTO TABLE_NAME
2. VALUES (value1, value2, value3, valueN);

For example:

1. INSERT INTO javatpoint (Author, Subject) VALUES ("Sonoo", "DBMS");

b. UPDATE: This command is used to update or modify the value of a column in the table.

Syntax:

1. UPDATE table_name SET [column_name1= value1,...column_nameN = valueN]
[WHERE CONDITION]

For example:

1. UPDATE students
2. SET User_Name = 'Sonoo'
3. WHERE Student_Id = '3'

c. DELETE: It is used to remove one or more row from a table.

Syntax:

1. DELETE FROM table_name [WHERE condition];

For example:

1. DELETE FROM javatpoint
2. WHERE Author="Albert";

DDL vs DML Commands

Explore the difference between DDL and DML commands in the below table. Understand how DDL commands shape database structures, while DML commands manipulate data within the database.

DDL**DML**

Used to define database objects like tables, indexes, views, etc.	Used to manipulate data within the database.
Examples of DDL statements include CREATE, ALTER, and DROP.	Examples of DML statements include SELECT, INSERT, UPDATE, and DELETE.
Changes made using DDL affect the structure of the database.	Changes made using DML affect the data stored in the database.

DDL statements are not transactional, meaning they cannot be rolled back.	DML statements are transactional, meaning they can be rolled back if necessary.
DDL statements are usually executed by a database administrator.	DML statements are executed by application developers or end-users.
DDL statements are typically used during the design and setup phase of a database.	DML statements are used during normal operation of a database.
Examples of DDL statements: CREATE TABLE, DROP TABLE, ALTER TABLE, CREATE INDEX, etc.	Examples of DML statements: SELECT, INSERT, UPDATE, DELETE, etc.

DCL (Data Control Language)

DCL includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions, and other controls of the database system.

List of DCL commands:

GRANT: This command gives users access privileges to the database.

Syntax:

```
GRANT SELECT, UPDATE ON MY_TABLE TO SOME_USER,  
ANOTHER_USER;
```

REVOKE: This command withdraws the user's access privileges given by using the GRANT command.

Syntax:

```
REVOKE SELECT, UPDATE ON MY_TABLE FROM USER1, USER2;
```

Advanced data manipulation using SQL

1. Data Transformation

Data obtained originally from a data source is often unprepared and messy. Data transformation involves some processes like modifying and converting data types, standardizing and normalizing data, and handling categorical data to make it fit for insightful data analysis.

Conversion of Data Types

Consider a “student” table that has a “score” column. It stores numerical values of “text” or VARCHAR type. In order to perform calculative analysis, you will need “integer” data types. SQL offers CONVERT and CAST functions for data type conversion:

— *Using **CONVERT** function*

```
SELECT name, CONVERT(INT, score) AS num_score  
  
FROM student_db;
```

2. SQL CASE Statements

SQL provides CASE statements that aid in logic building. These come in particularly handy when you want to retrieve and filter particular results based on certain conditions. CASE statements are also used to handle NULL values in data. This contributes to the consistency and reliability of the data.

Some example use cases are as follows:

Handling NULL values

Consider a table called “students,” where the “score” column contains several NULL values. You can replace the NULL values with a default score of 0.

```
SELECT score,  
      CASE WHEN score IS NULL  
      THEN 0  
      ELSE score  
      END AS new_score  
FROM students;
```

Handling categorical values

CASE statements are often used to handle categorical data.

Suppose the “students” table has another column “gender”, but with variations like “MALE”, “male”, “m”, “FEMALE”, “F”. This can make the data messy.

These values can be standardized as follows:

```
SELECT name,  
  
      CASE  
  
      WHEN UPPER(gender) = 'M'  
  
      OR UPPER(gender) = 'MALE' THEN 'Male'  
  
      WHEN UPPER(gender) = 'F'  
  
      OR UPPER(gender) = 'FEMALE' THEN 'Female'  
  
      ELSE 'Unknown'  
  
      END AS new_gender  
  
FROM students;
```

Handling conditional statements

Suppose you want to categorize student performance from the “score” column as ‘High,’ ‘Medium,’ or ‘Low’, based on their scores:

```
SELECT name, score,  
  
    CASE  
  
        WHEN score >= 70 THEN 'High'  
  
        WHEN score >= 40 THEN 'Medium'  
  
        ELSE 'Low'  
  
    END AS score_category  
  
FROM students;
```

3. Grouping data

SQL offers various functions for filtering and aggregating data to organize it according to a given criteria.

The GROUP BY function is used along with SQL aggregate functions to group similar rows and apply calculations to get the required results. Some common aggregate functions include SUM, AVG, COUNT, MIN, and MAX.

Data Aggregation:

— to calculate sum of values for each group

SELECT gender, SUM(score) AS total_score

FROM students

GROUP BY gender;

— to calculate average of values for each group

SELECT AVG(age) AS average_age

FROM students;

— to calculate total count for each group

SELECT age, COUNT() AS student_count*

FROM students

GROUP BY age;

— to find maximum values in a column

SELECT MAX(percentage) AS max_percentage

FROM students;

— to find minimum values in a column

SELECT MIN(percentage) AS min_percentage

FROM students;

4. SQL Unions

When combining multiple results based on similar sets—which may include the same columns, the same order, or similar data types—you use the SQL UNION clause.

While UNION ALL permits duplicate records, the UNION clause only returns unique records.

Let's say there are two tables in a database, student_p and student_q, with the same set of columns and different records, and you want to combine distinct rows from both of the tables:

```
SELECT name, age, score
```

```
FROM student_p
```

```
UNION
```

```
SELECT name, age, score
```

```
FROM student_q;
```

5. Handling Time Series Data

Time series data includes `TIMESTAMP`, `DATE`, and `DATETIME` values, which are used to keep track of time. Formatting time-based data is very crucial, as it can give you an insightful analysis of data based on time. Time-based indexing improves query performance and aids in query optimization for huge datasets.

To obtain time-based summary statistics such as daily, weekly, hourly, or annual results, SQL aggregate functions such as `SUM`, `AVG`, and `COUNT` are used.

```
SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount) AS
total_sales

FROM sales

GROUP BY month

ORDER BY month;
```

6. JOIN Queries

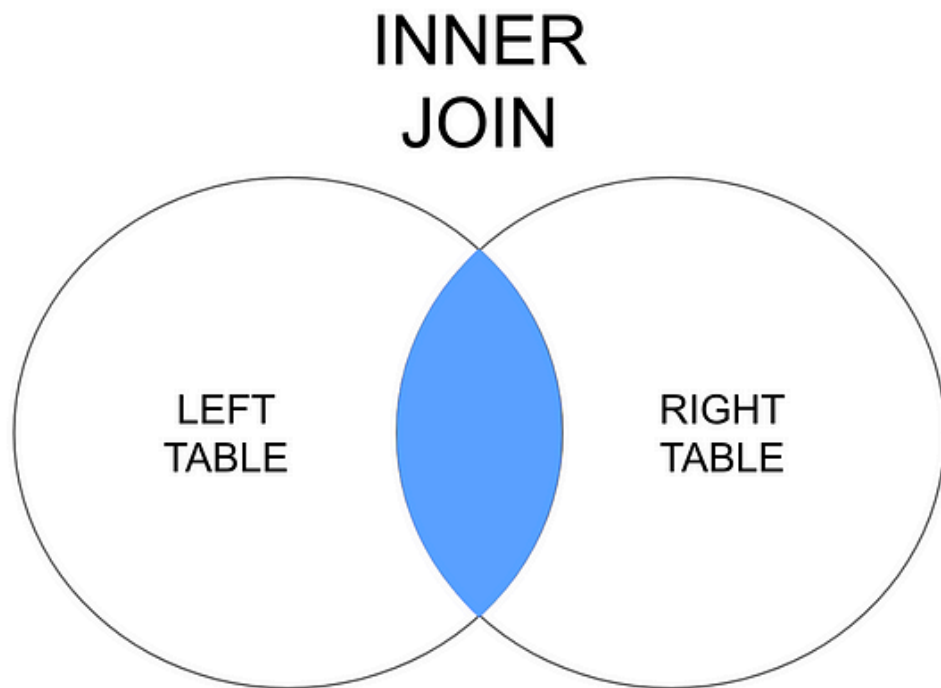
Let's create another table called `Products`. This is a table where the students will buy the product.

```
create table products (
    id int,
    name varchar(100),
    student_id int,
    primary key (id)
);

insert into products values (1, 'LEGO', 2);
insert into products values (2, 'LEGO', 5);
insert into products values (3, 'Toy Train', 5);
insert into products values (4, 'Toy Train', 8);
insert into products values (5, 'Shirt', 10);
```

A JOIN Query is used when we have to query a column from a second table. Normally there is a key from the first table that equates to a key in the second table. It can be in the form of an ID or string.

- INNER JOIN



Inner Join is joining one or more tables and selecting the rows as long as there is a match between both tables. It is like an INTERSECT.

```
SELECT s.first_name, s.last_name, st.name as sport
FROM students s
INNER JOIN sport_types st
on s.sport_id = st.id
where s.age = 17
```


In this select statement, we are joining the students table with the sports table on the sports_id (id in sport_types and sport_id in students) and filtering by students age (17). Notice I am using the **ALIAS** function, where I reference the students table as s and sport_types table as st. With this line:

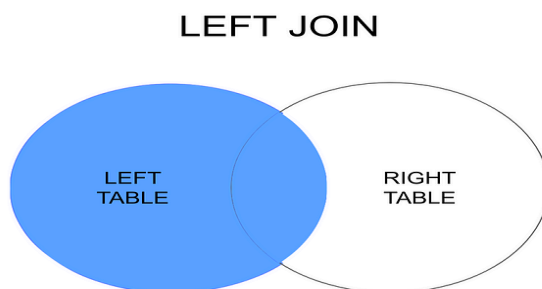
```
FROM students s
INNER JOIN sport_types st
on s.sport_id = st.id
```

I am joining the students table with the sport_types table on the sport_id column for students and the id for sport_types. If they have the same value, SQL will return the rows.

Result:

first_name	last_name	sport
Jack	Blaine	Soccer
Jane	Bale	Diving
James	Logan	Hockey
Clay	Powers	Frisbee
Evan	Parker	Water Polo
Maria	Hill	Basketball

- LEFT JOIN

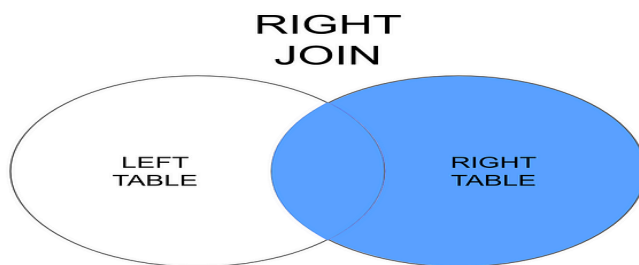


LEFT JOIN selects all records from the left table and records that match the right table.

```
SELECT s.first_name, s.last_name, p.name  
From students s  
LEFT JOIN products p  
on p.student_id = s.id
```

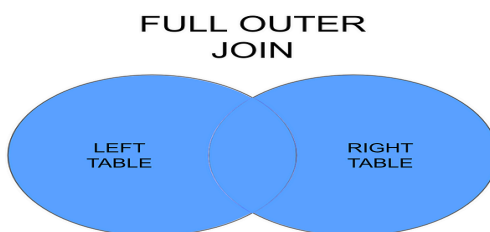
This query will return all the records of the left table (students) regardless of whether `p.name` is present. Notice that in that field `p.name`, it is null if there is no record of it.

- RIGHT JOIN



RIGHT JOIN selects all records from the right table and records that match the left table.

- FULL OUTER JOIN



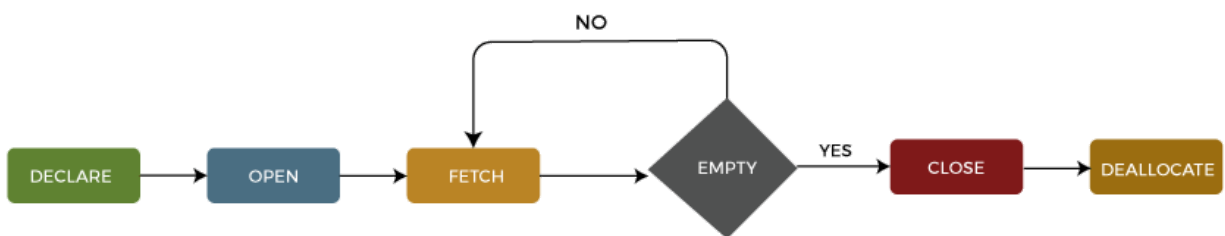
FULL OUTER JOIN selects all records for both the left and right tables.

Database Programming: Cursor

Cursor is a Temporary Memory or Temporary Work Station. It is Allocated by Database Server at the Time of Performing DML(Data Manipulation Language) operations on the Table by the User. Cursors are used to store Database Tables.

Life Cycle of the cursor

We can describe the life cycle of a cursor into the five different sections as follows:



1: Declare Cursor

The first step is to declare the cursor using the below SQL statement:

1. DECLARE cursor_name CURSOR
2. FOR select_statement;

We can declare a cursor by specifying its name with the data type CURSOR after the DECLARE keyword. Then, we will write the SELECT statement that defines the output for the cursor.

2: Open Cursor

It's a second step in which we open the cursor to store data retrieved from the result set. We can do this by using the below SQL statement:

1. OPEN cursor_name;

3: Fetch Cursor

It's a third step in which rows can be fetched one by one or in a block to do data manipulation like insert, update, and delete operations on the currently active row in the cursor. We can do this by using the below SQL statement:

1. `FETCH NEXT FROM cursor INTO variable_list;`

We can also use the `@@FETCHSTATUS` function in SQL Server to get the status of the most recent `FETCH` statement cursor that was executed against the cursor. The `FETCH` statement was successful when the `@@FETCHSTATUS` gives zero output. The `WHILE` statement can be used to retrieve all records from the cursor. The following code explains it more clearly:

1. `WHILE @@FETCH_STATUS = 0`
2. `BEGIN`
3. `FETCH NEXT FROM cursor_name;`
4. `END;`

4: Close Cursor

It's a fourth step in which the cursor should be closed after we finished work with a cursor. We can do this by using the below SQL statement:

1. `CLOSE cursor_name;`

5: Deallocate Cursor

It is the fifth and final step in which we will erase the cursor definition and release all the system resources associated with the cursor. We can do this by using the below SQL statement:

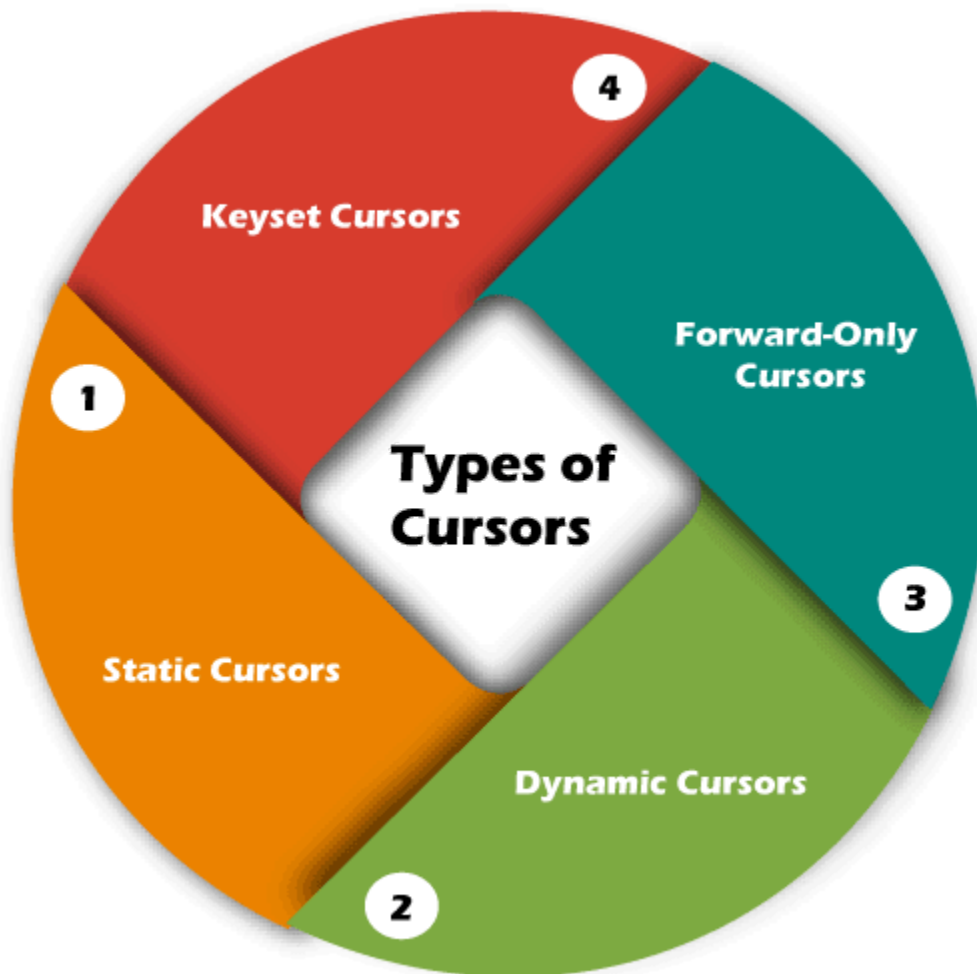
1. `DEALLOCATE cursor_name;`

Types of Cursors in SQL Server

The following are the different types of cursors in SQL Server listed below:

- Static Cursors
- Dynamic Cursors

- Forward-Only Cursors
- Keyset Cursors



Static Cursors

The result set shown by the static cursor is always the same as when the cursor was first opened. Since the static cursor will store the result in tempdb, they are always read-only. We can use the static cursor to move both forward and backward. In contrast to other cursors, it is slower and consumes more memory. As a result, we can use it only when scrolling is necessary, and other cursors aren't suitable.

This cursor shows rows that were removed from the database after it was opened. A static cursor does not represent any INSERT, UPDATE, or DELETE operations (unless the cursor is closed and reopened).

Dynamic Cursors

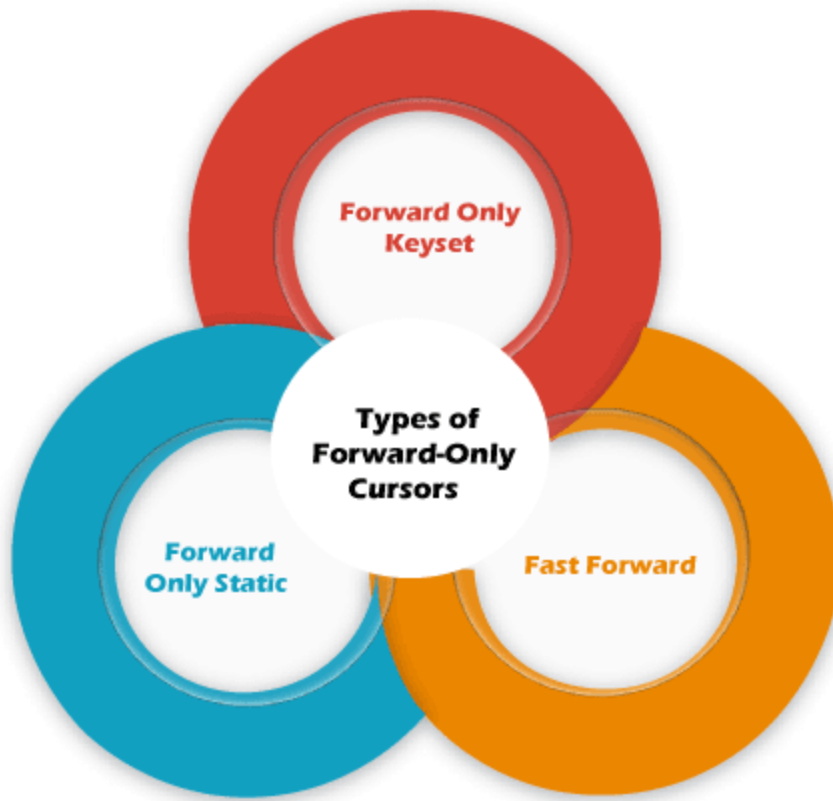
The dynamic cursors are opposite to the static cursors that allow us to perform the data updation, deletion, and insertion operations while the cursor is open. It is scrollable by default. It can detect all changes made to the rows, order, and values in the result set, whether the changes occur inside the cursor or outside the cursor. Outside the cursor, we cannot see the updates until they are committed.

Forward-Only Cursors

It is the default and fastest cursor type among all cursors. It is called a forward-only cursor because it moves only forward through the result set. This cursor doesn't support scrolling. It can only retrieve rows from the beginning to the end of the result set. It allows us to perform insert, update, and delete operations. Here, the effect of insert, update and delete operations made by the user that affect rows in the result set are visible as the rows are fetched from the cursor. When the row was fetched, we cannot see the changes made to rows through the cursor.

The Forward-Only cursors are three categorize into three types:

1. Forward_Only Keyset
2. Forward_Only Static
3. Fast_Forward



Keyset Driven Cursors

This cursor functionality lies between a static and a dynamic cursor regarding its ability to detect changes. It can't always detect changes in the result set's membership and order like a static cursor. It can detect changes in the result set's rows values as like a dynamic cursor. It can only move from the first to last and last to the first row. The order and the membership are fixed whenever this cursor is opened.

It is operated by a set of unique identifiers the same as the keys in the keyset. The keyset is determined by all rows that qualified the SELECT statement when the cursor was first opened. It can also detect any changes to the data source, which supports update and delete operations. It is scrollable by default.

Fetch Data from the Cursor There is a total of 6 methods to access data from the cursor. They are as follows:

1. FIRST is used to fetch only the first row from the cursor table.
2. LAST is used to fetch only the last row from the cursor table.

3. NEXT is used to fetch data in a forward direction from the cursor table.
4. PRIOR is used to fetch data in a backward direction from the cursor table.
5. ABSOLUTE n is used to fetch the exact nth row from the cursor table.
6. RELATIVE n is used to fetch the data in an incremental way as well as a decremental way.

SQL Cursor Exceptions

Whenever we execute an SQL query then there is the possibility of an error that is unexpected. Cursor goes through each set of rows to return in an SQL query.

There are some very popular exceptions:

1. Duplicate Value: This type of error occur when the cursor tries to insert a record or tuple which already exists in the database. these types of errors can be avoided by handling proper error conf
2. Invalid Cursor State: Whenever the cursor is in an invalid state this type of error will show as an error.
3. Lock Timeout: This occurs when the cursor tries to obtain a lock on a row or table but the lock is already held by another transaction.

Need of Cursor in an SQL server

1. Cursors allow us to process data row-by-row, which can be useful when we need to perform complex calculations or transformations on the data.
2. Cursors allow us to iterate over a result set multiple times, which can be useful when we need to perform multiple operations on the same data.

3. Cursors can be useful when we need to join multiple tables with complex relationships, such as when processing hierarchical data structures or when performing recursive queries.
4. Cursors allow us to perform operations such as updating, deleting, or inserting records based on some condition or criteria.
5. Cursors are especially useful when processing data from multiple tables where the relationships are not straightforward.

SQL Server Cursor Limitations

As a cursor has some limitations, it should only be used when there is no other choice. These restrictions include:

1. When processing data, it imposes locks on a subset or the entire table.
2. The cursor updates table records one row at a time, which slows down its performance.
3. While loops are slower than cursors, they do have more overhead.
4. Another factor that influences cursor speed is the quantity of rows and columns brought into the cursor.

Trigger

A trigger is a stored procedure in a database that automatically invokes whenever a special event in the database occurs. For example, a trigger can be invoked when a row is inserted into a specified table or when specific table columns are updated in simple words a trigger is a collection of SQL statements with particular names that are stored in system memory. It belongs to a specific class of stored procedures that are automatically invoked in response to database server events. Every trigger has a table attached to it.

The following are the key differences between triggers and stored procedures:

1. Triggers cannot be manually invoked or executed.
2. There is no chance that triggers will receive parameters.
3. A transaction cannot be committed or rolled back inside a trigger.

Syntax:

```
create trigger [trigger_name]  
[before | after]  
{insert | update | delete}  
on [table_name]  
[for each row]  
[trigger_body]
```

Explanation of Syntax

1. Create trigger [trigger_name]: Creates or replaces an existing trigger with the trigger_name.
2. [before | after]: This specifies when the trigger will be executed.
3. {insert | update | delete}: This specifies the DML operation.
4. On [table_name]: This specifies the name of the table associated with the trigger.

5. [for each row]: This specifies a row-level trigger, i.e., the trigger will be executed for each affected row.
6. [trigger_body]: This provides the operation to be performed as the trigger is fired

Why Do We Employ Triggers?

When we need to carry out some actions automatically in certain desirable scenarios, triggers will be useful. For instance, we need to be aware of the frequency and timing of changes to a table that is constantly changing. In such cases, we could create a trigger to insert the required data into a different table if the primary table underwent any changes.

Different Trigger Types in SQL Server

Two categories of triggers exist:

1. DDL Trigger
2. DML Trigger
3. Logon Triggers

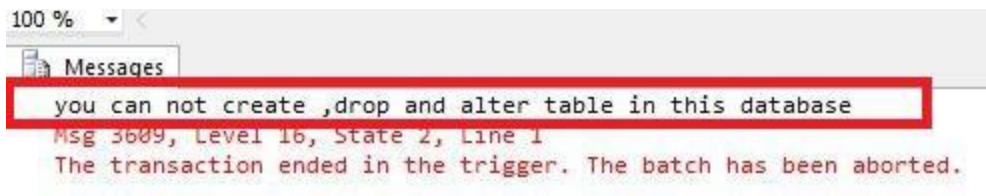
DDL Triggers

The Data Definition Language (DDL) command events such as Create_table, Create_view, drop_table, Drop_view, and Alter_table cause the DDL triggers to be activated.

SQL Server

```
create trigger safety
on database
for
create_table,alter_table,drop_table
as
print 'you can not create,drop and alter tab
```

Output:



100 % <

Messages

you can not create ,drop and alter table in this database
Msg 3609, Level 16, State 2, Line 1
The transaction ended in the trigger. The batch has been aborted.

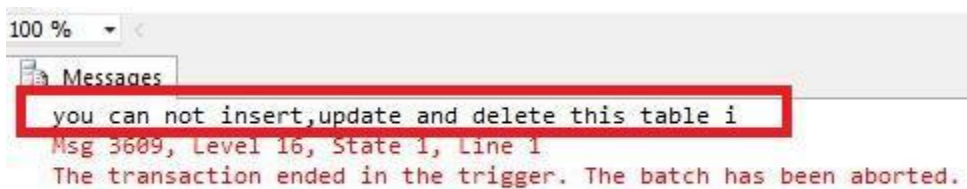
DML Triggers

The Data uses manipulation Language (DML) command events that begin with Insert, Update, and Delete set off the DML triggers. corresponding to insert_table, update_view, and delete_table.

SQL Server

```
create trigger deep
on emp
for
insert,update ,delete
as
print 'you can not insert,update and delete this table i'
rollback;
```

Output:



100 % <

Messages

you can not insert,update and delete this table i
Msg 3609, Level 16, State 1, Line 1
The transaction ended in the trigger. The batch has been aborted.

Logon Triggers

logon triggers are fires in response to a LOGON event. When a user session is created with a SQL Server instance after the authentication process of logging is finished but before establishing a user session, the LOGON event takes place. As a result, the

PRINT statement messages and any errors generated by the trigger will all be visible in the SQL Server error log. Authentication errors prevent logon triggers from being used. These triggers can be used to track login activity or set a limit on the number of sessions that a given login can have in order to audit and manage server sessions.

Advantage of Triggers

The benefits of using triggers in SQL Server include the following:

1. Database object rules are established by triggers, which cause changes to be undone if they are not met.
2. The trigger will examine the data and, if necessary, make changes.
3. We can enforce data integrity thanks to triggers.
4. Data is validated using triggers before being inserted or updated.
5. Triggers assist us in maintaining a records log.
6. Due to the fact that they do not need to be compiled each time they are run, triggers improve the performance of SQL queries.
7. The client-side code is reduced by triggers, saving time and labor.
8. Trigger maintenance is simple.

Disadvantage of Triggers

The drawbacks of using triggers in SQL Server include the following:

1. Only triggers permit the use of extended validations.
2. Automatic triggers are used, and the user is unaware of when they are being executed. Consequently, it is difficult to troubleshoot issues that arise in the database layer.
3. The database server's overhead may increase as a result of triggers.
4. In a single CREATE TRIGGER statement, we can specify the same trigger action for multiple user actions, such as INSERT and UPDATE.

5. Only the current database is available for creating triggers, but they can still make references to objects outside the database.